Case Study 01: ATM Transaction Sequence Simulation

BASIC I/O & ARITHMETIC

1 PROBLEM STATEMENT

Simulate an Automated Teller Machine (ATM) transaction sequence. The system must prompt for an initial balance, take a withdrawal amount, verify sufficiency of funds, and compute the remaining balance.

2 OBJECTIVES

- Practise user input parsing and numeric operations.
- Implement conditional validation for transaction approval.

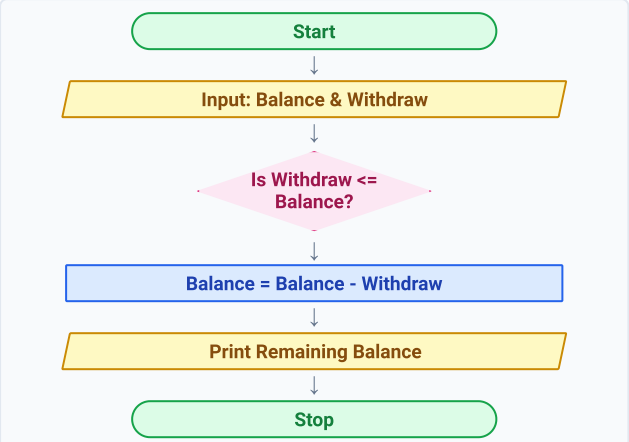
3 ASSUMPTIONS

Initial balance is positive. Input values are valid positive numerical numbers. No daily limits applied.

4 ALGORITHM

1. Initialize starting balance (e.g., Rs. 5000).
2. Input withdrawal amount.
3. Check if withdrawal amount ≤ balance.
4. Subtract withdrawal amount from balance.
5. Display remaining balance and transaction status.

5 VERTICAL FLOWCHART



6 PSEUDOCODE

```
BEGIN
  SET balance = 5000
  READ withdraw_amount
  IF withdraw_amount <= balance THEN
    balance = balance - withdraw_amount
    PRINT "Transaction Successful"
  ELSE
    PRINT "Insufficient Funds"
  ENDIF
  PRINT balance
END
```

7 PYTHON SOURCE CODE

```python
# ATM Withdrawal Simulator
balance = 5000.0
withdraw = 1200.0

if withdraw <= balance:
    balance -= withdraw
    status = "APPROVED"
else:
    status = "DECLINED"

print(f"Status: {status}")
print(f"Remaining Balance: Rs. {balance:.2f}")
```

8 TEST CASES

#	Input	Expected Output
1	Bal: 5000, W/D: 1200	Status: APPROVED, Rem: 3800.00
2	Bal: 5000, W/D: 6000	Status: DECLINED, Rem: 5000.00

9 OUTPUT SCREENSHOT

```
TERMINAL / OUTPUT CONSOLE
=== ATM TRANSACTION REPORT ===
Initial Balance : Rs. 5000.00
Withdrawal Req : Rs. 1200.00
Status : APPROVED
Remaining Bal : Rs. 3800.00
```

10 CONCLUSION

The program successfully models standard ATM transaction logic, preventing overdrafts through explicit condition checking.

Case Study 02: Student Performance Trend Analysis

DATA STRUCTURES & AGGREGATIONS

1 PROBLEM STATEMENT

Analyze student subject scores to evaluate overall academic performance by computing the sum, mean score, and performance category.

2 OBJECTIVES

- Utilize Python lists for sequential data storage.
- Calculate statistical parameters (Total & Average).

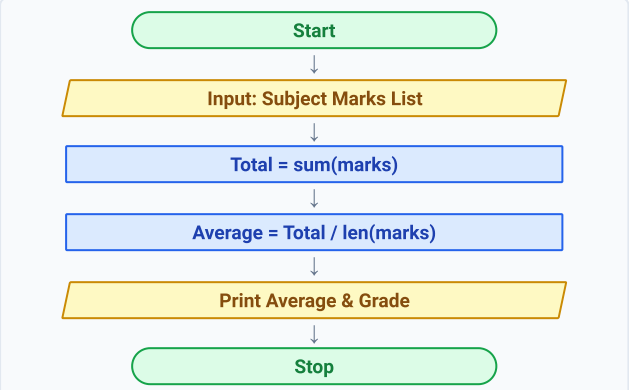
3 ASSUMPTIONS

Marks are given out of 100 per subject. Non-negative numeric inputs are assumed for all evaluated subjects.

4 ALGORITHM

1. Store subject marks in a list structure.
2. Sum all elements in the marks list.
3. Divide the total sum by list length to find the average.
4. Determine grade criteria based on calculated mean.
5. Print detailed evaluation summary.

5 VERTICAL FLOWCHART



6 PSEUDOCODE

```
BEGIN
  SET marks = [78, 85, 90]
  SET total = SUM(marks)
  SET average = total / LENGTH(marks)
  IF average >= 80 THEN grade = "Distinction"
  ELSE grade = "Pass"
  PRINT total, average, grade
END
```

7 PYTHON SOURCE CODE

```python
# Student Performance Analysis
marks = [78, 85, 90]
total = sum(marks)
average = total / len(marks)

grade = "Distinction" if average >= 80 else "Pas
s"

print(f"Total Marks : {total}")
print(f"Average     : {average:.2f}")
print(f"Grade       : {grade}")
```

8 TEST CASES

#	Input List	Expected Output
1	[78, 85, 90]	Total: 253, Avg: 84.33, Distinction
2	[50, 60, 55]	Total: 165, Avg: 55.00, Pass

9 OUTPUT SCREENSHOT

```
TERMINAL / OUTPUT CONSOLE
=== PERFORMANCE REPORT ===
Marks Recorded : [78, 85, 90]
Total Marks : 253
Average Score : 84.33
Performance : Distinction
```

10 CONCLUSION

The algorithm effectively leverages built-in list functions to automate standard statistical evaluation of student academic marks.

Case Study 03: Advanced Numeric Pattern Generator

ITERATION & LOOPS

1 PROBLEM STATEMENT

Generate a structured sequence mapping integers from 1 to 5 to their respective mathematical squares using automated loop iteration.

2 OBJECTIVES

- Master `for` loop construction and `range()` limits.
- Perform inline power operations during iteration.

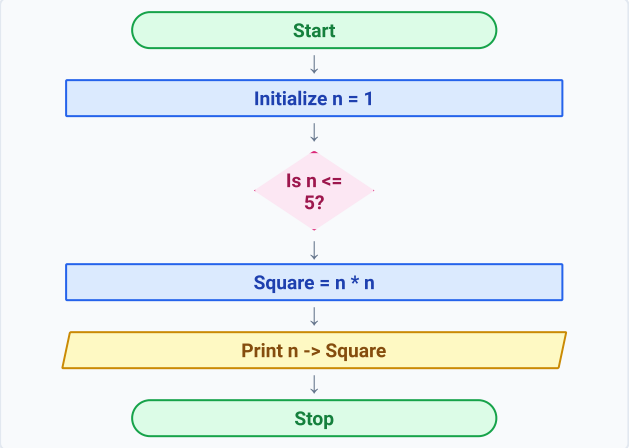
3 ASSUMPTIONS

Lower bound is inclusive (1), upper bound is inclusive (5). Arithmetic exponentiation is standard base-10.

4 ALGORITHM

1. Initialize a loop for range index `n` from 1 to 5.
2. Compute `square = n * n` for each step.
3. Format and print the index alongside its square.
4. Terminate loop upon reaching upper bound limit.

5 VERTICAL FLOWCHART



6 PSEUDOCODE

```
BEGIN
  FOR n FROM 1 TO 5 DO
    SET square = n * n
    PRINT n, "->", square
  ENDFOR
END
```

7 PYTHON SOURCE CODE

```python
# Numeric Pattern Generator
print("N  ->  Square")
print("----------------")
for n in range(1, 6):
    square = n * n
    print(f"{n}  ->  {square}")
```

8 TEST CASES

#	Range Input	Expected Sequence Output
1	range(1, 6)	1->1, 2->4, 3->9, 4->16, 5->25
2	range(1, 3)	1->1, 2->4

9 OUTPUT SCREENSHOT

```
TERMINAL / OUTPUT CONSOLE
=== NUMBER SQUARES ===
1 -> 1
2 -> 4
3 -> 9
4 -> 16
5 -> 25
```

10 CONCLUSION

The code demonstrates deterministic looping logic, efficiently processing and transforming numeric ranges in repetitive sequences.

Case Study 04: Password Policy Validation Engine

STRING HANDLING & LOGIC

1 PROBLEM STATEMENT

Validate user-submitted passwords against basic security rules. The system evaluates whether string length meets the 8-character minimum policy threshold.

2 OBJECTIVES

- Understand string length operations via `len()`.
- Apply branch logic to grant or deny access.

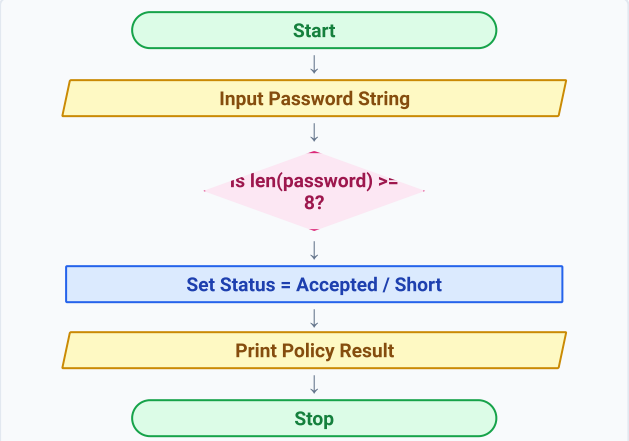
3 ASSUMPTIONS

Passwords can contain any mix of alphanumeric and special characters. Input string spaces count toward character length.

4 ALGORITHM

1. Receive password string input.
2. Calculate length of string using built-in function.
3. If length ≥ 8, classify status as "ACCEPTED".
4. Else, classify status as "REJECTED (Too Short)".
5. Output validation message.

5 VERTICAL FLOWCHART



6 PSEUDOCODE

```
BEGIN
  READ password
  IF LENGTH(password) >= 8 THEN
    PRINT "Password Accepted"
  ELSE
    PRINT "Error: Too short!"
  ENDIF
END
```

7 PYTHON SOURCE CODE

```python
# Password Policy Checker
password = "Study@123"

if len(password) >= 8:
    result = "Password Accepted"
else:
    result = "Too Short! Min 8 chars required."

print(f"Input    : {password}")
print(f"Validation: {result}")
```

8 TEST CASES

#	Input Password	Expected Output
1	"Study@123" (len 9)	Password Accepted
2	"Pass1" (len 5)	Too Short! Min 8 chars required.

9 OUTPUT SCREENSHOT

```
TERMINAL / OUTPUT CONSOLE
=== PASSWORD CHECK ===
Entered String : Study@123
String Length : 9 characters
Result : Password Accepted
```

10 CONCLUSION

The system enforces basic authentication hygiene by filtering non-compliant short string inputs using string evaluation methods.

Case Study 05: Text Compression & Character Frequency Count

DICTIONARIES & HASH MAPS

1 PROBLEM STATEMENT

Count occurrence frequencies of each distinct character in a given string. This operation serves as the core foundation for text compression and encoding algorithms.

2 OBJECTIVES

- Understand key-value mapping with Python dictionaries.
- Safely update dictionary entries using `.get()`.

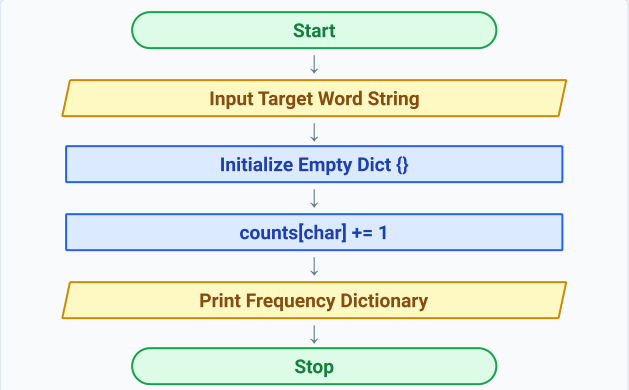
3 ASSUMPTIONS

Input text is converted to lowercase. Case sensitivity is ignored for fundamental letter run counting.

4 ALGORITHM

1. Initialize target string (e.g. "banana").
2. Create an empty frequency dictionary.
3. Iterate through each character in string.
4. Increment count in dictionary by 1.
5. Print complete frequency table.

5 VERTICAL FLOWCHART



6 PSEUDOCODE

```
BEGIN
  SET word = "banana"
  SET counts = EMPTY DICTIONARY
  FOR EACH char IN word DO
    counts[char] = GET(counts, char, 0) + 1
  ENDFOR
  PRINT counts
END
```

7 PYTHON SOURCE CODE

```python
# Character Frequency Counter
word = "banana"
counts = {}

for ch in word:
    counts[ch] = counts.get(ch, 0) + 1

print(f"Target Word : {word}")
print(f"Frequencies : {counts}")
```

8 TEST CASES

#	Input Word	Expected Output Dictionary
1	"banana"	{'b': 1, 'a': 3, 'n': 2}
2	"apple"	{'a': 1, 'p': 2, 'l': 1, 'e': 1}

9 OUTPUT SCREENSHOT

```
TERMINAL / OUTPUT CONSOLE
=== CHARACTER COUNTS ===
Word Analyzed : banana
Frequency Map : {'b': 1, 'a': 3, 'n': 2}
```

10 CONCLUSION

Dictionary mapping allows O(N) execution efficiency when profiling character occurrences in text payloads.

Case Study 06: Student Marks Matrix Analysis

2D ARRAYS & NESTED LISTS

1 PROBLEM STATEMENT

Process a 2D matrix storing mark records for multiple students across multiple exams, generating row-wise total scores for each student.

2 OBJECTIVES

- Manage multi-dimensional data structures in Python.
- Iterate through rows and perform aggregate calculations.

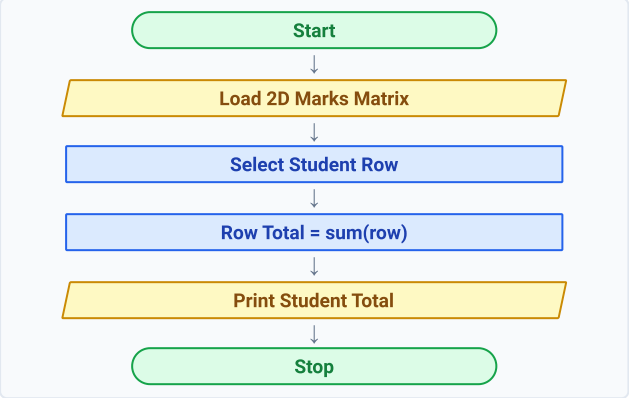
3 ASSUMPTIONS

Each row represents an individual student. Each column represents a specific exam score.

4 ALGORITHM

1. Define 2D list containing score vectors.
2. Iterate through rows with an index counter.
3. Compute `sum()` of current row values.
4. Display individual student totals.

5 VERTICAL FLOWCHART



6 PSEUDOCODE

```
BEGIN
  SET matrix = [[80, 75, 90], [70, 85, 88]]
  FOR idx, row IN ENUMERATE(matrix) DO
    SET total = SUM(row)
    PRINT "Student", idx+1, "Total:", total
  ENDFOR
END
```

7 PYTHON SOURCE CODE

```python
# 2D Matrix Marks Processing
marks_matrix = [
    [80, 75, 90],  # Student 1
    [70, 85, 88]   # Student 2
]

for i, row in enumerate(marks_matrix, 1):
    row_total = sum(row)
    print(f"Student {i} Total: {row_total}")
```

8 TEST CASES

#	Input Matrix	Expected Row Totals
1	[[80, 75, 90], [70, 85, 88]]	S1: 245, S2: 243
2	[[10, 20], [30, 40]]	S1: 30, S2: 70

9 OUTPUT SCREENSHOT

```
TERMINAL / OUTPUT CONSOLE
=== MARKS MATRIX ANALYSIS ===
Student 1 Marks: [80, 75, 90]
Student 1 Total: 245
Student 2 Marks: [70, 85, 88]
Student 2 Total: 243
```

10 CONCLUSION

Nested structures efficiently encapsulate multi-attribute records, enabling automated sub-total processing across row boundaries.

Case Study 07: Tuple-Based Product Catalogue Audit

IMMUTABLE TUPLES & INDEXING

1 PROBLEM STATEMENT

Retrieve, format, and audit product inventory details bound within immutable tuples to guarantee data integrity during operational catalog queries.

2 OBJECTIVES

- Store related heterogeneous values in Python tuples.
- Access record fields using positional zero-based indexing.

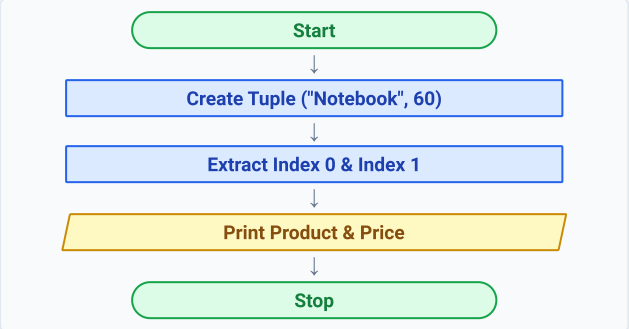
3 ASSUMPTIONS

Tuple order is fixed: Index 0 stores Product Name (str), Index 1 stores Price in INR (float).

4 ALGORITHM

1. Construct product tuple e.g. `("Notebook", 60)`.
2. Extract name from `product[0]`.
3. Extract price from `product[1]`.
4. Display audit statement formatted for invoice printing.

5 VERTICAL FLOWCHART



6 PSEUDOCODE

```
BEGIN
  SET product = ("Notebook", 60)
  SET name = product[0]
  SET price = product[1]
  PRINT "Product:", name
  PRINT "Price: Rs.", price
END
```

7 PYTHON SOURCE CODE

```python
# Tuple Product Catalogue
product = ("Notebook", 60.0)

name = product[0]
price = product[1]

print(f"Audit Item : {name}")
print(f"Unit Price : Rs. {price:.2f}")
```

8 TEST CASES

#	Tuple Input	Expected Output
1	("Notebook", 60)	Item: Notebook, Price: Rs. 60.00
2	("Pen", 15)	Item: Pen, Price: Rs. 15.00

9 OUTPUT SCREENSHOT

```
TERMINAL / OUTPUT CONSOLE
=== CATALOGUE AUDIT ===
Record Tuple : ('Notebook', 60.0)
Audit Item : Notebook
Unit Price : Rs. 60.00
```

10 CONCLUSION

Tuples provide lightweight, immutable data containers ideal for protecting critical system master data from unauthorized dynamic mutation.

Case Study 08: Food Waste Monitoring System

RESOURCE TRACKING & METRICS

1 PROBLEM STATEMENT

Track food consumption in a school dining facility by computing remaining plates post-service and determining surplus/waste percentages.

2 OBJECTIVES

- Apply basic arithmetic subtraction and division in Python.
- Monitor waste metrics for operational efficiency.

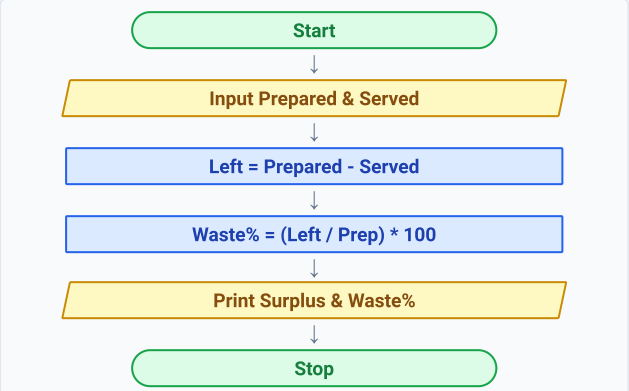
3 ASSUMPTIONS

Prepared food quantity is greater than or equal to served food quantity. Measured in discrete plate units.

4 ALGORITHM

1. Set quantity of prepared plates.
2. Set quantity of served plates.
3. Calculate `left = prepared - served`.
4. Calculate `waste_percent = (left / prepared) * 100`.
5. Output waste metrics.

5 VERTICAL FLOWCHART



6 PSEUDOCODE

```
BEGIN
  SET prepared = 50
  SET served = 42
  SET left = prepared - served
  SET waste_pct = (left / prepared) * 100
  PRINT left, waste_pct
END
```

7 PYTHON SOURCE CODE

```python
# Food Waste Tracker
prepared = 50
served = 42

left = prepared - served
waste_pct = (left / prepared) * 100

print(f"Prepared : {prepared} plates")
print(f"Served   : {served} plates")
print(f"Surplus  : {left} plates ({waste_pct:.1
f}%)")
```

8 TEST CASES

#	Inputs (Prep, Serv)	Expected Output
1	50, 42	Surplus: 8 plates (16.0%)
2	100, 95	Surplus: 5 plates (5.0%)

9 OUTPUT SCREENSHOT

```
TERMINAL / OUTPUT CONSOLE
=== FOOD WASTE MONITOR ===
Food Prepared : 50 plates
Food Served : 42 plates
Food Leftover : 8 plates (16.0%)
```

10 CONCLUSION

Automated resource calculations provide actionable intelligence to reduce kitchen over-preparation and minimize food waste.

Case Study 09: Multi-Product Price Fluctuation Analysis

FINANCIAL CALCULATION

1 PROBLEM STATEMENT

Calculate price differentials between baseline historical prices and updated market prices, classifying movements as inflationary or deflationary.

2 OBJECTIVES

- Compute net variance between numerical variables.
- Evaluate directional movement using conditional statements.

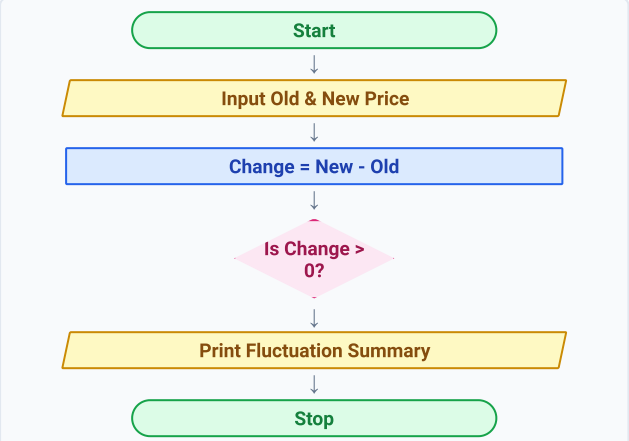
3 ASSUMPTIONS

Prices are denominated in Indian Rupees (INR). Currency values are positive floating-point numbers.

4 ALGORITHM

1. Store baseline price and updated price.
2. Compute `change = new_price - old_price`.
3. If change > 0, tag as "Increase".
4. Else if change < 0, tag as "Decrease".
5. Display formatted variance report.

5 VERTICAL FLOWCHART



6 PSEUDOCODE

```
BEGIN
  SET old_price = 100
  SET new_price = 115
  SET change = new_price - old_price
  IF change > 0 THEN trend = "Increase"
  ELSE trend = "Decrease"
  PRINT change, trend
END
```

7 PYTHON SOURCE CODE

```python
# Price Fluctuation Analyzer
old_price = 100.0
new_price = 115.0

change = new_price - old_price
trend = "Increase" if change > 0 else "Decrease"

print(f"Baseline Price : Rs. {old_price:.2f}")
print(f"Updated Price  : Rs. {new_price:.2f}")
print(f"Variance       : Rs. {abs(change):.2f} {t
rend}")
```

8 TEST CASES

#	Inputs (Old, New)	Expected Output
1	100.0, 115.0	Variance: Rs. 15.00 Increase
2	200.0, 180.0	Variance: Rs. 20.00 Decrease

9 OUTPUT SCREENSHOT

```
TERMINAL / OUTPUT CONSOLE
=== PRICE CHANGE REPORT ===
Old Price : Rs. 100.00
New Price : Rs. 115.00
Net Variance : Rs. 15.00 Increase
```

10 CONCLUSION

The system evaluates dynamic cost shifts, calculating precise metric differences for inventory pricing governance.

Case Study 10: Smart Canteen Order & Stock Management

INVENTORY CONTROLS

1 PROBLEM STATEMENT

Manage cafeteria inventory by checking whether available stock satisfies incoming order quantities before fulfilling requests and adjusting balances.

2 OBJECTIVES

- Implement relational comparison checks (`<=`).
- Maintain inventory consistency via state updating.

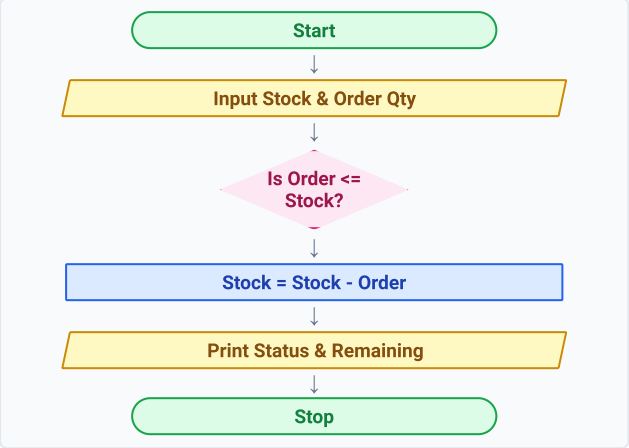
3 ASSUMPTIONS

Stock levels cannot drop below zero. Orders are processed atomically as whole integer units.

4 ALGORITHM

1. Load baseline available stock.
2. Read customer order quantity.
3. If `order_qty <= stock`, approve order & deduct stock.
4. Else, reject order due to stock deficiency.
5. Print inventory management report.

5 VERTICAL FLOWCHART



6 PSEUDOCODE

```
BEGIN
  SET stock = 20
  SET order_qty = 6
  IF order_qty <= stock THEN
    stock = stock - order_qty
    PRINT "Order Accepted"
  ELSE
    PRINT "Insufficient Stock"
  ENDIF
  PRINT stock
END
```

7 PYTHON SOURCE CODE

```python
# Canteen Stock Manager
stock = 20
order_qty = 6

if order_qty <= stock:
    stock -= order_qty
    status = "Order Accepted"
else:
    status = "Insufficient Stock"

print(f"Status          : {status}")
print(f"Remaining Stock : {stock}")
```

8 TEST CASES

#	Input (Stock, Order)	Expected Output
1	Stock: 20, Order: 6	Status: Accepted, Remaining: 14
2	Stock: 10, Order: 15	Status: Insufficient Stock, Remaining: 10

9 OUTPUT SCREENSHOT

```
TERMINAL / OUTPUT CONSOLE
=== SMART CANTEEN INVENTORY ===
Initial Stock : 20 items
Order Request : 6 items
Order Status : Accepted
Updated Stock : 14 items remaining
```

10 CONCLUSION

Conditional checking prevents negative stock balances, ensuring point-of-sale reliability across retail environments.